Falas por Integrante v6 (9,5 minutos)

Mimir — Gestão de Documentos para IA na AWS | Alinhada à apresentação com o case Mimir (17 slides, 4 integrantes)

Equipe: Athos, Gabriel, Fabricio, Lucas

> **Como usar**
> Cada integrante tem a própria página: dá para imprimir só a sua. Itálico laranja é indicação de cena e não se fala. Destaque azul é frase de impacto: diga com pausa antes e depois.

Integrante	Slides	Tempo	Palavras
Athos	Slides 1 a 5	2:00	~277
Fabricio	Slides 6 a 9	2:00	~264
Gabriel	Slides 10 a 14	2:00	~289
Lucas	Slides 15 a 17	3:45	~304

> **Tempo total**
> 1134 palavras, cerca de 8.1 minutos de fala a 140 palavras por minuto, mais ~1,5 minuto de cliques da demo: cerca de 9:30 no total. Ensaiem com cronômetro: se passar, o primeiro corte é o passo do Deep Archive na demo (~30 s).

> **O que mudou em relação à v5**
>
> - O case agora se chama Mimir: "Startup XYZ" e "XYZ" saíram das falas.
> - Novo slide 2, "Mimir e a nossa arquitetura", na fala do Athos (~30 s): a lenda da fonte de Mímir ligada ao S3, aos documentos, ao STS e ao histórico que treina a IA.
> - A conclusão do Lucas fecha a história do nome: a fonte só dá visão de futuro se for bem guardada.
> - Numeração atualizada para 17 slides: Athos 1 a 5, Fabricio 6 a 9, Gabriel 10 a 14, Lucas 15 a 17.
> - Para caber no tempo, saíram: a frase "não construímos um repositório de arquivos" (Athos), a abertura das "três perguntas" e a frase do antimalware (Gabriel) e o Glacier Instant na evolução (Lucas).

Athos 2:00 min (~277 palavras)

Slides 1 a 5 — Capa, Mimir e a nossa arquitetura, Entendimento do Problema, Levantamento de Requisitos e Visão Geral da Solução.

[olhar para a banca, não para o slide, ao dizer o número]

Bom dia. Nós somos Athos, Gabriel, Fabricio e Lucas, e este é o case Mimir. Quero deixar um número com vocês: ***cinquenta mil***. É a quantidade de documentos que a Mimir recebe por mês, de clientes que confiam a ela arquivos que não podem ser perdidos nem vistos por outra empresa.

[transição de slide: Mimir e a nossa arquitetura]

[tom de história, sem ler a tabela]

O nome vem de Mímir, o ser da mitologia nórdica que guarda a fonte de toda a sabedoria. Odin deu um olho para beber dela e ganhar visão de futuro.

Na nossa arquitetura, a fonte é o bucket S3, e cada documento enviado é uma gota. Só bebe quem se autentica e recebe uma credencial presa à própria pasta. E o histórico acumulado é o que dá visão de futuro à IA.

Agora, o problema que essa fonte resolve.

[transição de slide: Entendimento do Problema]

A Mimir é um SaaS que extrai dados de PDFs e imagens com IA. Quando ela cresce, aparecem três problemas: se todo upload passar pelo servidor, ele vira gargalo; o cliente A nunca pode ver o documento do cliente B; e guardar tudo em armazenamento rápido faz o custo subir todo mês.

[transição de slide: Levantamento de Requisitos]

Separamos os requisitos em dois grupos. Funcionais: autenticar, receber sem gargalo, isolar por cliente, baixar com segurança e preservar o histórico. Não funcionais: escalar, respeitar a LGPD, custo proporcional ao uso, alta disponibilidade sem administrar servidores e auditoria.

[transição de slide: Visão Geral da Solução]

Em linguagem simples: o usuário faz login e recebe um token; a API confere esse token; o sistema devolve uma chave temporária que abre só aquele arquivo; e o arquivo vai direto para o armazenamento, que depois de um ano o move sozinho para uma camada muito mais barata. O Fabricio mostra como cada peça faz isso.

Fabricio 2:00 min (~264 palavras)

Slides 6 a 9 — Diagrama de Arquitetura, Serviços AWS, Justificativa Técnica e Atendimento aos Requisitos.

[apontar o diagrama da esquerda para a direita, seguindo os números das setas]

Obrigado, Athos. O jeito óbvio seria um servidor ligado o tempo todo esperando uploads. Decidimos não fazer isso: com pouco mais de um arquivo por minuto, ele ficaria ocioso quase sempre e ainda seria o gargalo nos picos.

[seta 1: Cliente → Cognito]

O login é feito no Amazon Cognito, que emite um token JWT. Não construímos login do zero.

[seta 2: Cliente → API Gateway]

Com o token, o app chama o API Gateway, que valida o JWT e limita o número de requisições. Sem token válido, a Lambda nem executa.

[setas 3 e 4: Lambda e STS]

Aqui está o diferencial: a Lambda não recebe o arquivo. Ela lê o usuário no token, pede ao STS uma credencial temporária que só enxerga a pasta desse usuário e assina com ela uma URL válida por quinze minutos.

O arquivo vai direto do cliente para o S3. A Lambda nunca toca no binário.

[apontar para o bucket S3 e para os quadros de baixo]

No S3, os objetos são cifrados com uma chave nossa no KMS, e CloudWatch e CloudTrail registram o que acontece. O GuardDuty aparece como adicional opcional.

[transição de slide: Justificativa Técnica]

[apontar a tabela]

Cada escolha tem uma alternativa descartada. Serverless em vez de EC2, porque não há servidor ocioso. Um bucket com uma pasta por cliente em vez de um bucket por cliente, porque seriam milhares de políticas, ciclos de vida e chaves para manter. E São Paulo em vez de Virgínia: Virgínia sairia cerca de quarenta e cinco por cento mais barata, mas dados de clientes brasileiros ficam no Brasil, pela LGPD.

[transição de slide: Atendimento aos Requisitos]

E cada requisito que o Athos apresentou tem um componente respondendo por ele, como mostra a tabela. O Gabriel agora mostra como essa arquitetura se comporta quando cresce, quando falha e quanto custa.

Gabriel 2:00 min (~289 palavras)

Slides 10 a 14 — Escalabilidade, Resiliência, Segurança, Custos e Backup e Recuperação.

[pausa de um segundo antes de começar, tom mais firme]

Crescimento. Cinquenta mil documentos por mês são pouco mais de um por minuto; dez vezes mais, doze por minuto. Não existe servidor para redimensionar: os limites que existem são cotas de conta, ajustáveis com um pedido à AWS. Na demo, cada autorização levou cerca de duzentos e cinquenta milissegundos.

Crescer dez vezes não exige redesenhar a solução.

[transição de slide: Resiliência]

Falhas. Não há instância para cair: Lambda, API Gateway e Cognito rodam em várias zonas, e o S3 grava cada objeto em pelo menos três zonas antes de confirmar o upload, por isso o RPO é praticamente zero. Não replicamos para outra região porque o Brasil tem uma só, e copiar dados pessoais para fora do país é transferência internacional pela LGPD.

[transição de slide: Segurança]

Segurança em três camadas. Identidade: a Lambda lê o usuário do token, nunca do que o cliente envia. Rede: sem VPC de propósito; o controle está no bucket fechado ao público, no TLS obrigatório e no limite de requisições. Dados: criptografia com KMS e CloudTrail registrando cada credencial emitida.

[transição de slide: Custos]

[apontar para a tabela de custos]

Custo. Em São Paulo, cerca de vinte e seis dólares por mês no primeiro ano e trinta e cinco no terceiro. Sem o ciclo de vida, o terceiro ano custaria setenta e quatro.

O armazenamento fica noventa e dois por cento mais barato no momento em que o documento deixa de ser operação e vira histórico.

O GuardDuty ficou opcional porque custaria uns vinte dólares por mês, quase o mesmo que toda a infraestrutura do primeiro ano.

[transição de slide: Backup e Recuperação]

Por fim, recuperação: o versionamento desfaz qualquer sobrescrita, e só a conta root apaga uma versão de vez. E um ponto que nós mesmos identificamos: falta DeletionPolicy Retain na chave KMS, e isso já está nos próximos passos. O Lucas agora mostra tudo funcionando.

Lucas 3:45 min (~304 palavras)

Slides 15 a 17 — Demonstração ao vivo, Evolução e Conclusão (inclui ~1,5 min de cliques da demo).

[tela: página da demo aberta em http://localhost:8000 (não 127.0.0.1), cliente-a já logado; console da Lambda em outra aba]

Agora vocês vão ver a arquitetura rodando, na nossa conta AWS em São Paulo.

[clicar em "Chamar a API sem login"]

Alguém de fora chama a API sem login: 401. Com um token forjado, 401 de novo. Isso para no API Gateway; a Lambda nem executa.

[escolher contrato-cliente-a.pdf e clicar em Enviar]

O cliente A envia um contrato. A API devolveu a URL e o navegador gravou direto no S3: 204. Reparem na chave: o arquivo caiu na pasta usuario- seguida do identificador desse usuário no Cognito, com um nome gerado pelo servidor.

[colar no campo 4 a chave do arquivo do cliente-b e clicar em Tentar baixar]

Agora o ataque: o cliente A pede um arquivo do cliente B. A aplicação recusa: 403. Mas e se o nosso código tivesse um bug?

[console: APP_CHECK = false, Salvar; voltar à página e clicar de novo em Tentar baixar]

Desliguei a checagem. A Lambda assinou a URL, e mesmo assim o S3 respondeu 403 AccessDenied, porque a credencial do STS só alcança a pasta do cliente A. ***O isolamento não depende de o código lembrar de checar.***

[console: voltar APP_CHECK para true e salvar]

[na lista, clicar em Baixar no documento DEEP_ARCHIVE e depois em Restaurar]

Por último, um documento em Deep Archive, como estaria depois de um ano: o download é recusado e o cliente pede a restauração, 202. Em até doze horas ele volta. Ninguém apagou nada.

[transição de slide: Evolução]

Tudo o que vocês viram resolve o problema de hoje. Os próximos passos: uma fila SQS entre o upload e a IA para absorver picos, Object Lock para tornar o histórico imutável e o Macie para encontrar dados pessoais antes do treino dos modelos.

[transição de slide: Conclusão]

[olhar para todos os avaliadores]

Para fechar, volto à fonte de Mímir: cinquenta mil gotas por mês que não podem ser perdidas nem vistas por outro cliente. E a fonte só dá visão de futuro se for bem guardada.

A resposta não é a arquitetura mais complexa. É a mais honesta com o problema: cada serviço faz o que faz de melhor, o histórico fica noventa e dois por cento mais barato e o isolamento entre clientes foi provado ao vivo.

Obrigado, e ficamos à disposição para as perguntas.